Ders 11 Genişletilmiş Notu: B-Ağaçları (Özellikler, Yükseklik, Arama, Ekleme, Silme) ve Yığınlara Giriş

Ders 11 Genişletilmiş Notu: B-Ağaçları (Özellikler, Yükseklik, Arama, Ekleme, Silme) ve Yığınlara Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. B-Ağaçlarının Hatırlatılması
- 2. B-Ağacının Özellikleri
- 3. Örnek: Ağaç Hangi t Değerleri İçin Geçerli?
- 4. B-Ağacının Yüksekliği
- 5. Arama: B-TREE-SEARCH
- 6. Boş Ağaç Oluşturma: B-TREE-CREATE
- 7. Düğüm Bölme: B-TREE-SPLIT-CHILD
- 8. Ekleme: B-TREE-INSERT ve B-TREE-INSERT-NONFULL
- 9. Ekleme Örneği ($t = 3$)
- 10. B-Ağacından Silme
- 11. Yığın (Heap)
- 12. Yığın Özelliğini Korumak: MAX-HEAPIFY
- 13. Yığın Oluşturma: BUILD-MAX-HEAP
- Örnek Sorular

1. B-Ağaçlarının Hatırlatılması

Hoca derse geçen hafta başlanan B-ağaçlarını (B-tree) hatırlatarak başladı. B-ağaçları disk işlemleri için, yani diskteki dosyalara hızlı erişmek ve arama yapmak için kullanılan bir yapıdır. İkili ağaçlardan farkı, bir düğümün birden fazla çocuğu olabilmesidir. Kırmızı-siyah ağaçlar gibi dengeli bir yapıları vardır; ekleme ve silme işlemleri ağacın dengeli kalmasını sağlayacak biçimde yapılmalıdır.

Soru: Bir düğümün çocuk sayısı sabit mi? (Soru kayıtta duyulmuyor, hocanın cevabından anlaşılıyor.) **Cevap:** Değişebilir. Bir t sabiti belirlenir ve çocuk sayısının bu t'ye göre bir en küçük ve en büyük değeri vardır; ağaç bu sınırları sağlamalıdır.

Her düğümde `x.n` anahtar ve `x.n + 1` çocuk vardır. Bir düğümdeki eleman sayısı diskin sayfa (page) boyutuyla orantılı seçilir; böylece her düğüm tek bir disk erişimiyle okunabilir. İşlemlerde kökün ana bellekte (main memory) tutulduğu varsayılır.

2. B-Ağacının Özellikleri

Hoca B-ağacının bütün işlemlerde korunması gereken özelliklerini tahtaya yazdı (ekrandan, 00:10:00). Herhangi bir x düğümü için:

1. x'in `x.n` adlı bir alanı (attribute) vardır: x düğümünde tutulan anahtarların sayısı (number of keys stored in x).
2. Anahtarlar $x.key_1, x.key_2, \ldots, x.key_{x.n}$ biçimindedir ve düğüm içinde sıralıdır: $x.key_1 \leq x.key_2 \leq \ldots \leq x.key_{x.n}$. Ayrıca x'in `x.leaf` adlı mantıksal (boolean) bir alanı vardır: x

yapraksa `TRUE`, iç düğümse (internal node) `FALSE`. Her iç düğümün `x.n + 1` çocuğu vardır ve bunlara $x.c_1, x.c_2, \ldots, x.c_{x.n+1}$ işaretçileriyle bağlanır.
3. Anahtarlar, her alt ağaçta tutulan anahtarların aralıklarını birbirinden ayırır (separate the ranges of keys stored in each subtree).
4. Bütün yapraklar aynı derinliktedir (all leaves have the same depth). Bu derinlik ağacın yüksekliği h ile gösterilir.
5. $t \geq 2$ olan bir sabit belirlenir. Bu sabite B-ağacının en küçük derecesi (minimum degree) denir.

Üçüncü özelliği hoca şekille açıkladı (ekrandan, 00:10:00). x düğümünde üç anahtar olsun ($x.n = 3$): $x.key_1$, $x.key_2$, $x.key_3$. O zaman dört çocuk vardır: $x.c_1, \ldots, x.c_4$. k_1, k_2, k_3, k_4 sırasıyla bu çocukların alt ağaçlarındaki herhangi birer anahtar olsun (her çocukta tek anahtar olması gerekmez, bunlar temsilîdir). Aralarındaki ilişki şöyledir:

$$k_1 \leq x.key_1 \leq k_2 \leq x.key_2 \leq k_3 \leq \ldots$$

Beşinci özellik, düğümlerdeki anahtar ve çocuk sayılarına iki sınır koyar:

- **En az:** Kök dışındaki her düğümün en az $t-1$ anahtarı olmalıdır. Bu yüzden her iç düğümün en az t çocuğu olur. Kök daha az anahtara sahip olabilir: en az 1 anahtar (root must have at least one key). Yani kök tek anahtarla kalabilir, diğer düğümler kalamaz. En basit B-ağacında da $t \geq 2$'dir.
- **En fazla:** Her düğümün en fazla $2t-1$ anahtarı (maximum number of keys) olabilir; dolayısıyla her iç düğümün en fazla $2t$ çocuğu olur. Bu üst sınır kök için de geçerlidir. Tam $2t-1$ anahtarı olan bir düğüme dolu (full) denir; ona daha fazla anahtar eklenemez.

3. Örnek: Ağaç Hangi t Değerleri İçin Geçerli?

Hoca geçen hafta çizdiği örnek ağacı tekrar çizdi (ekrandan, 00:13:50). Kökte yalnız M vardır. M'den küçükler soldaki D H düğümüne, büyükler sağdaki üç anahtarlı düğüme gider. Yapraklar soldan sağa B C, F G, J K L, N P, R S, V W, Y Z'dir.

> **Ders sonrası not:** Hoca tahtada sağdaki iç düğümü "Q T S" diye yazdı (altyazıda da "T ve S" geçiyor). Çocukları R S, V W ve Y Z olduğuna göre üçüncü anahtar, önceki dersteki çizimde olduğu gibi X olmalıdır (Q T X). Aşağıdaki soru yalnız anahtar sayılarına baktığı için cevap değişmez.

Hoca sordu: Yazılan kurallara göre bu ağaç t'nin hangi değerleri için geçerli bir B-ağacıdır?

Çözüm sınıfla birlikte kuruldu. Kökte bir anahtar vardır; köke buna zaten izin verilir. Diğer düğümlerde iki ya da üç anahtar vardır.

- $t = 2$: Her düğümde en az $t-1 = 1$, en fazla $2t-1 = 3$ anahtar olmalıdır. Bütün düğümlerde 1 ile 3 arasında anahtar vardır; ağaç geçerlidir.
- $t = 3$: En az $t-1 = 2$, en fazla $2t-1 = 5$ anahtar olmalıdır. Bütün düğümlerde 2 ile 5 arasında anahtar vardır; ağaç geçerlidir.
- $t = 1$: Olamaz, çünkü $t \geq 2$ olmalıdır.
- $t = 4$: En az $t-1 = 3$ anahtar gerekir; ama iki anahtarlı düğümler vardır. Geçerli değildir.

Sonuç: ağaç yalnız $t = 2$ ve $t = 3$ için geçerlidir.

4. B-Ağacının Yüksekliği

Hoca ardından bir teorem kanıtladı (ekrandan, 00:20:00):

> Eğer $n \geq 1$ ise, yüksekliği h ve en küçük derecesi $t \geq 2$ olan, n anahtarlı her B-ağacı için
>
> $$h \leq \log_t \frac{n+1}{2}.$$

İkili arama ağaçlarında ikiye ayrılma olduğu için 2 tabanlı logaritmalardan söz ediliyordu. Burada t'ye ayrılma olduğu için t tabanlı logaritma çıkar.

Kanıtın fikri şudur: h'nin t ve n'ye bağlı bir değerden küçük olduğu gösterilecek. Ağacın en uzun olduğu durum nedir? Her düğümde daha az bilgi tutulursa ağaç mecburen uzar. Bu yüzden kökün ve düğümlerin mümkün olan en az anahtara sahip olduğu ağaç kurulur.

- Kökte 1 anahtar vardır. Tek anahtar olduğu için kökten yalnız 2 düğüm çıkar.
- Bu iki düğümün her birinde en az $t-1$ anahtar vardır. Her birinin t çocuğu olur.
- Bu çocukların her birinde yine $t-1$ anahtar ve t çocuk vardır; yapı böyle devam eder.

Hoca sınıftan her seviyedeki düğüm sayısını hesaplamasını istedi ve tabloyu birlikte doldurdu (ekrandan, 00:26:45):

Derinlik	Düğüm sayısı
0	1 (yalnız kök)
1	2
2	$2t$
3	$2t^2$
4	$2t^2 \cdot t = 2t^3$

Derinlik 2'de iki düğümün her birinin t çocuğu olduğundan $2t$ düğüm vardır. Derinlik 3'te bu $2t$ düğümün her birinin t çocuğu olduğundan $2t^2$ düğüm vardır. Genel olarak derinlik $i \geq 1$'de $2t^{i-1}$ düğüm bulunur.

Derinlik 1'den h'ye kadar her düğümde $t-1$ anahtar vardır; bunların toplamı $(t-1)\sum_{i=1}^{h} 2t^{i-1}$'dir. Kök istisnadır, onun tek anahtarı ayrıca eklenir. Anahtar sayısı en az bu kadardır; daha fazla da olabilir, çünkü düğümlerin tutabileceği en fazla anahtar $t-1$ değildir. Böylece eşitsizlik çıkar:

$$n \geq 1 + (t-1)\sum_{i=1}^{h} 2t^{i-1}$$

Hoca bunun ikinci derslerde verilen toplam formülleriyle çözüldüğünü söyledi. Geometrik seri toplamı uygulanınca (ara adım eklendi):

$$n \geq 1 + 2(t-1)\cdot\frac{t^h - 1}{t-1} = 2t^h - 1$$

Buradan $n+1 \geq 2t^h$ olur. Her iki tarafın t tabanında logaritması alınınca

$$\log_t \frac{n+1}{2} \geq h$$

elde edilir; bu kanıtlanmak istenen şeydir. ($1 + (t-1)\sum_{i=1}^{h} 2t^{i-1} = 2t^h - 1$ eşitliği $2 \leq t \leq 7$, $0 \leq h \leq 7$ için python ile doğrulandı.)

5. Arama: B-TREE-SEARCH

Arama bir noktadan, kökten başlatılır. Bu düğüm x, aranan anahtar k'dir. Hoca sözde kodu yazarak yapının nasıl işlediğini de gösterdi.

İkili arama ağacında bir düğüme bakılır, anahtar büyükse sağa, küçükse sola gidilir ve dallanarak aşağı inilirdi; arama ağacın derinliği boyunca sürerdi. B-ağacında yalnız aşağı inmek yetmez; her düğümün içinde de sağa doğru tarama yapmak gerekir. Yani önce düğümün içinde aranır, sonra ilgili çocuğa dallanılır. Hoca düğüm içi aramayı `while` döngüsüyle, kökten yapraklara doğru aramayı özyinelemeli (recursive) olarak yazdı (ekrandan, 00:31:15):

```
B-TREE-SEARCH(x, k)
i = 1
while i <= x.n and k > x.key_i
    i = i + 1
if i <= x.n and k == x.key_i
    return (x, i)
elseif x.leaf
    return NIL
else DISK-READ(x.c_i)
    return B-TREE-SEARCH(x.c_i, k)
```

- `while` döngüsü $k \leq x.key_i$ olan en küçük i'yi bulur: anahtarları tek tek tarar, bulamadıkça i'yi artırır. k düğümdeki bütün anahtarlardan büyükse döngüden $i = x.n + 1$ ile çıkılır.
- i hâlâ anahtarların içinde bir yerdeyse ve $x.key_i = k$ ise anahtar bulunmuştur; `(x, i)` döner.
- x yapraksa (`x.leaf`) anahtar ağaçta yoktur; `NIL` döner.
- Değilse bir disk işlemi yapılmalıdır. İkili ağaçtaki yordamdan farkı budur: bellekte yalnız bir düğüm tutulabildiği için dallanılacak sonraki çocuk diskten okunur (`DISK-READ`), sonra aynı fonksiyon o çocuk için özyinelemeli çağrılır.

Soru (hoca sordu): Bu yordamın karmaşıklığı nedir? **Cevap:** `while` döngüsü bir düğümü baştan sona tarar. Bir düğümde en fazla $2t-1$ anahtar olabildiği için bu kısım $O(t)$ getirir. Özyinelemeli kısım ağacın derinliğiyle ilgilidir ve $O(\log_t n)$ getirir. Toplam (ekrandan, 00:33:45):

$$O(t \log_t n)$$

Ek bilgi (derste anlatılmadı): Ders kitabı bu sonucu iki ayrı ölçüyle verir: arama sırasında erişilen disk sayfası sayısı $O(h) = O(\log_t n)$'dir, işlemci (CPU) süresi ise her düğümde $O(t)$ iş yapıldığı için $O(t \log_t n)$'dir. Disk tabanlı yapılarda asıl belirleyici genelde disk erişim sayısıdır.

6. Boş Ağaç Oluşturma: B-TREE-CREATE

B-ağacını kurmak için yardımcı fonksiyonlar gerekir: boş bir düğüm oluşturan bir fonksiyon ve bu düğümle boş bir ağaç oluşturan bir fonksiyon. Hoca bunu da sözde kodla anlattı (ekrandan, 00:40:00):

```
B-TREE-CREATE(T)
x = ALLOCATE-NODE()
x.leaf = TRUE
x.n = 0
DISK-WRITE(x)
T.root = x
```

`ALLOCATE-NODE` dile özgü çalıştığı için hoca onun kodunu yazmadı. Bu fonksiyonun bir disk okuması içermesi gerekmez; yalnız yer ayırır, bir şey okumaz. Dersin başında anlatılan alanlardan `leaf` ve `n` ayarlanır: ağaç boş olduğu için düğüm yapraktır (`TRUE`) ve anahtar sayısı 0'dır. Bu bilgiler veri yapısına yazıldıktan sonra düğüm diske yazılır ve kök yapılır.

7. Düğüm Bölme: B-TREE-SPLIT-CHILD

İkili ağaçlarda ekleme daha kolaydı. B-ağacına eleman eklerken her düğümün alabileceği en fazla anahtar sayısı bellidir. Bu sınırın aşılma olasılığı varsa ilgili düğüm ikiye bölünmeli, işaretçiler ve ebeveyn bilgileri güncellenmelidir. Bunun için ekleme açısından kritik olan `B-TREE-SPLIT-CHILD` fonksiyonu yazılır.

7.1 Fikir

$2t-1$ anahtarlı (yani dolu) bir y düğümü olsun. Bölündüğünde iki ayrı düğüm oluşur ve her birinde $t-1$ anahtar bulunur. İki tarafın toplamı $2t-2$ olduğuna göre bir anahtar eksiktir: o, y'nin ortasındaki ortanca (median) anahtardır ve ebeveyne çıkar. Ebeveynde bu anahtar iki yeni düğümü birbirinden ayırır.

7.2 Sözde kod

Fonksiyon x ve i alır. Anlamı "x düğümünü i'inci anahtardan böl" değil, "x düğümünün i'inci çocuğunu böl"dür. Yani bölünecek düğümün kendisi değil, ebeveyni girdi olarak verilir; hoca bunun akılda tutulmasını istedi. Tahtadaki kod (ekrandan, 00:50:00):

```
B-TREE-SPLIT-CHILD(x, i)
z = ALLOCATE-NODE()
y = x.c_i
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
    z.key_j = y.key_{j+t}
if not y.leaf
    for j = 1 to t
        z.c_j = y.c_{j+t}
y.n = t - 1
for j = x.n + 1 downto i + 1
    x.c_{j+1} = x.c_j
x.c_{i+1} = z
for j = x.n downto i
    x.key_{j+1} = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Satır satır:

- Yeni bir düğüm oluşturulur ve z'ye atanır. Bölünecek çocuk $x.c_i$, y'de tutulur; sonraki işlemler y üzerinden yürür.
- y yapraksa z de yaprak olur, değilse olmaz.
- z'de saklanacak anahtar sayısı $t-1$'dir.
- y'nin t'den sonraki anahtarları, yani $t+1, t+2, \ldots, 2t-1$ indisli anahtarları z'nin ilk $t-1$ anahtarı yapılır. Hoca bunu birinci sınıftaki dizilerde kaydırma işlemlerine benzetti.
- Yalnız anahtarları taşımak yetmez; y yaprak değilse çocukları da taşınır. Çocuk sayısı bir fazla olduğu için bu döngü $t-1$'e değil t'ye kadar gider.
- y'nin anahtar sayısı $t-1$ olarak güncellenir.
- y'nin ortanca anahtarı ebeveyne çıkacağı için x'te yer açılmalıdır. x'in çocukları, $x.n+1$'den $i+1$'e kadar birer sağa kaydırılır ve aradaki boşluğa z bağlanır ($x.c_{i+1} = z$).
- Aynı biçimde x'in anahtarları birer sağa kaydırılır ve ortanca anahtar $y.key_t$, $x.key_i$ olarak yukarı çıkarılır.
- x'in anahtar sayısı bir artırılır ve değişen üç düğüm diske yazılır.

7.3 Şekil üzerinde

Hoca $t = 4$ için bir şekil çizdi (ekrandan, 00:50:00). $t = 4$ olduğundan bir düğümün en fazla $2t = 8$ çocuğu olabilir. x düğümünde N ve W gibi anahtarlar vardır ($x.key_{i-1}$ ve $x.key_i$ civarı). $y = x.c_i$ dolu düğümdür: P Q R S T U V; çocukları $T_1, \ldots, T_8$'dir.

1. Yeni z düğümü oluşturulur.
2. y'nin son kısmı T U V, z'ye taşınır; çocukları da kendi döngüsüyle birlikte taşınır.
3. Ortanca anahtar S, x'te N ile W arasına yazılır; x'in yeni işaretçisi z'yi gösterir.
4. Anahtar sayıları güncellenir.

Sonuçta x'te … N S W … vardır. S'den küçükler (P Q R) bir tarafta, S'den büyükler (T U V) diğer taraftadır. Hoca bu şekillerin hepsinin dersin başında kaynak olarak verdiği kitapta bulunduğunu ve oradan da takip edilebileceğini söyledi.

8. Ekleme: B-TREE-INSERT ve B-TREE-INSERT-NONFULL

8.1 B-TREE-INSERT

T ağacına k anahtarı eklenecek (ekrandan, 01:13:32):

```
B-TREE-INSERT(T, k)
r = T.root
if r.n == 2t - 1
    s = ALLOCATE-NODE()
    T.root = s
    s.leaf = FALSE
    s.n = 0
    s.c_1 = r
    B-TREE-SPLIT-CHILD(s, 1)
    B-TREE-INSERT-NONFULL(s, k)
else B-TREE-INSERT-NONFULL(r, k)
```

Önce kökten başlanır. Kökün anahtar sayısı $2t-1$ ise kök doludur ve önce bölünmelidir. Bunun için yeni bir s düğümü ayrılır ve kök yapılır; eski kök r artık kök değildir. s yaprak değildir (`FALSE`), şu an hiç anahtarı yoktur, ama bir çocuğu vardır: r. Sonra `B-TREE-SPLIT-CHILD(s, 1)` ile s'nin birinci çocuğu, yani r bölünür. Ardından `B-TREE-INSERT-NONFULL(s, k)` çağrılır. Kök dolu değilse aynı fonksiyon doğrudan r ile çağrılır: bölme yapıldıysa ekleme s üzerinden, yapılmadıysa r üzerinden başlar.

Hoca bunu da şekille gösterdi (ekrandan, 01:13:32). $t = 4$; dolu kök r'de A D F H L N P (7 anahtar) ve 8 çocuk ($T_1, \dots, T_8$) vardır.

1. Yeni s düğümü kök yapılır, r onun çocuğu olur.
2. `SPLIT-CHILD` yeni bir düğüm oluşturur ve r'nin son yarısını, L N P anahtarlarını çocuklarıyla birlikte oraya taşır.
3. r'nin ortancası H, s'ye yazılır ve yeni düğüm s'nin çocuğu yapılır.
4. Sonuç: kök s = H; çocukları A D F (r) ve L N P.

8.2 B-TREE-INSERT-NONFULL

Hoca bu fonksiyonun biraz kritik olduğunu söyledi. Fonksiyon, kendisine gelen x düğümünün dolu olmadığını kabul ederek k'yi x'e ekler; aslında dolu olmamayı garanti edecek biçimde çalışır. Tahtadaki kod (ekrandan, 01:20:00 ve 01:22:35):

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
    while i >= 1 and k < x.key_i
        x.key_{i+1} = x.key_i
        i = i - 1
    x.key_{i+1} = k
    x.n = x.n + 1
    DISK-WRITE(x)
else while i >= 1 and k < x.key_i
        i = i - 1
    i = i + 1
    DISK-READ(x.c_i)
    if x.c_i.n == 2t - 1
        B-TREE-SPLIT-CHILD(x, i)
        if k > x.key_i
            i = i + 1
    B-TREE-INSERT-NONFULL(x.c_i, k)
```

Yaprak durumu. i, x'in anahtar sayısıyla başlar. x yapraksa k'nin yaprakta yerleşmesi gereken yer bulunmalıdır. Aramadaki gibi döngüyle dönülür: $i \geq 1$ ve $k < x.key_i$ olduğu sürece anahtarlar birer sağa kaydırılır, böylece eklenecek elemana yer açılır. Kaydırma bitince boşta kalan yere k yazılır, anahtar sayısı bir artırılır ve düğüm diske yazılır.

> **Hocanın vurgusu:** B-ağacında ekleme araya yapılmaz, yaprağa yapılır. Bu fonksiyon özyinelemelidir ve özyinelemeden çıkış noktası yaprağa gelinen durumdur. Fonksiyon, yaprağın dolu olmadığını, bir eleman daha alabileceğini garanti edecek biçimde tasarlanmıştır; bu yüzden yaprakta doluluk kontrolü yapılmaz.

İç düğüm durumu. x yaprak değilse hangi çocuğa dallanılacağı bulunmalıdır. Yine benzer bir döngüyle k'den küçük olan anahtar bulunur; k onun hemen sağındaki çocuğa gitmelidir, bu yüzden i bir artırılır. Dallanılacak çocuk diskten okunur. Sonra o çocuğun anahtar sayısının $2t-1$ olup olmadığına, yani çocuğun dolu olup olmadığına bakılır. Doluysa mecburen bölünür: `SPLIT-CHILD`'a x ve kaçıncı çocuğun bölüneceği gönderilir. Bölmeden sonra iki çocuk oluşmuştur; k'nin hangisine gideceği seçilmelidir. k, yukarı çıkan $x.key_i$'den büyükse sağdaki çocuğa gidilir (i bir artırılır), değilse aynı yerde kalınır. Son olarak uygun alt ağaca eklemek için `INSERT-NONFULL` özyinelemeli çağrılır.

Böylece fonksiyon inerken karşılaştığı dolu düğümleri böler ve yaprağa kadar gider; yaprakta yukarıdaki biçimde eklemeyi yapar. Hoca özetle her seferinde şu kontrollerin yapıldığını söyledi: kök dolu mu, ilgili çocuğa in, çocuk dolu mu, tekrar özyinele.

> **Ek bilgi (derste anlatılmadı):** Bu yaklaşımda ekleme ağaçta yalnız bir kez aşağı inerek tamamlanır; dolu düğümler önceden bölündüğü için ekleme sonrası yukarı doğru geri dönüp düzeltme yapmak gerekmez. Ağaç yalnız kök bölündüğünde, yani yukarıdan bir seviye büyür. Ders kitabı eklemenin $O(h)$ disk erişimi ve $O(th) = O(t \log_t n)$ işlemci süresi gerektirdiğini belirtir.

9. Ekleme Örneği ($t = 3$)

Hoca nasıl çalıştığını şekil üzerinde gösterdi. $t = 3$ olduğu için bir düğümde en fazla $2t-1 = 5$ anahtar olabilir (ekrandan, 01:24:20 ve 01:30:00).

Başlangıç ağacı: kökte G M P X vardır. G'den küçükler A C D E yaprağında, G ile M arasındakiler J K'de, M ile P arasındakiler N O'da, P ile X arasındakiler R S T U V'de, X'ten büyükler Y Z'dedir. Sırayla B, Q, L ve F eklenecek.

B ekleme. Kökten aranmaya başlanır ve her seferinde kökün dolu olup olmadığına bakılır. Kök dolu mu? Hayır (4 anahtar). İlgili çocuk A C D E; dolu mu? Hayır. B bu yaprağa eklenir: anahtarlar kaydırılır ve B, A ile C arasına yazılır. Yaprak A B C D E olur. Bu en basit durumdur.

Q ekleme. Kök dolu değildir. Q, P ile X arasında olmalıdır; ilgili çocuk R S T U V'dir ve doludur. Önce bölünür: R S bir düğüm, U V bir düğüm olur; ortanca T köke çıkar. Kök G M P T X olur. Artık Q, R S'nin bulunduğu yaprağa yazılabilir: Q R S.

L ekleme. Önce kök kontrol edilir: G M P T X doludur. Kök bölünmek zorundadır: G M bir parça, T X bir parça olur, ortadaki P yukarı alınır ve yeni kök olur. G M'nin üç çocuğu (A B C D E, J K, N O), T X'in üç çocuğu (Q R S, U V, Y Z) vardır. L, J K'ye gider ve yaprak J K L olur.

F ekleme. Kök P dolu değildir. F, P'nin sol tarafındadır; G M'ye inilir, dolu değildir. Buradan F'nin gideceği yaprak A B C D E'ye inilir; bu yaprakta beş anahtar vardır, yani doludur. Bu kez eklenecek yaprak dolu olduğu için o bölünür: ortanca C yukarı çıkar ve G M düğümü C G M olur. Yapraklar A B ve D E olur. Bağlantılar güncellenir: C ile G arasındaki işaretçi D E'yi gösterir. Artık F'nin gideceği yaprak dolu değildir ve F eklenir: D E F.

Son ağaç (ekrandan, 01:30:00):

```
                  P
        C G M             T X
A B  D E F  J K L  N O   Q R S  U V  Y Z
```

Hoca kodun bu örnek üzerinde satır satır izlenebileceğini önerdi. (Dört ekleme python ile yazılan bir B-ağacı eklemesiyle çalıştırıldı; her adım ve son ağaç tahtadakiyle aynıdır.)

10. B-Ağacından Silme

Silme işlemi daha karmaşıktır. Kitapta ve farklı kaynaklarda silme için izlenmesi gereken bir dizi kural vardır. Silme de özyinelemeli yapılır; özyinelemeli çözüm kolaydır. "Şu durum olursa

şöyle, bu durum olursa böyle" biçiminde tanımlanmış kurallar koda gömülür. Hoca kodu yazmadı; kitaptaki şekil üzerinden örneklerle açıkladı (ekrandan, 01:35:05 ve 01:37:28).

Eklerken $2t - 1$'in aşılmamasına dikkat ediliyordu. Silerken de $t - 1$'in altına düşülmemesine dikkat edilir. Bu kontrol yalnız ilgili çocuk için değil, onun kardeşleri, ebeveyni ve ebeveyninin kardeşleri için de yapılır.

Örnekte $t = 3$'tür. Başlangıç ağacı, ekleme örneğinin sonundaki ağaçtır: kök P; iç düğümler C G M ve T X; yapraklar A B, D E F, J K L, N O, Q R S, U V, Y Z. Şekildeki adımlar sırasıyla şöyledir:

1. **F silinir (şekilde "case 1").** En basit durum yapraktan silmektir. F silinmeden önce durumuna bakılır; F yapraktan doğrudan kaldırılır ve D E kalır. Sorun yoktur.
2. **M silinir ("case 2a").** Geçen derste ikili arama ağacında öncülün (predecessor) nasıl bulunduğu gösterilmişti: M'den küçük en büyük eleman. Burada M'nin öncülü L'dir. L ile M'nin yeri değiştirilir: M'nin yerine L konur, böylece M kaldırılır ve L de yapraktan çıkarılır. İç düğüm C G L, yaprak J K olur.
3. **G silinir ("case 2c").** G'nin iki çocuğu (D E ve J K) en az anahtar sayısındadır. Bu yüzden ikisi birleştirilir: G silinir ve C ile L arasında kalan iki düğüm D E J K olarak birleşir. C ile L arasındaki işaretçi bu düğümü gösterecek biçimde ayarlanır. İç düğüm C L olur.
4. **D silinir ("case 3b").** D silinirken ebeveynlere bakılır. C L ve T X düğümleri en az anahtar sayısında olduğu için bu kardeş düğümler birleştirilir; kökteki P bunların ortancası olarak ortaya alınır: C L P T X. Kök boşalır ve ağacın yüksekliği bir azalır (şekilde "tree shrinks in height"). D, E J K'nin bulunduğu yapraktan doğrudan çıkarılır: E J K.
5. **B silinir ("case 3a").** Burada silinen anahtarın bulunduğu düğümle birlikte kardeşlerinin de anahtar sayısına bakılır. A B en az sayıdadır (2), kardeşi E J K'de ise 3 anahtar vardır. Bu yüzden birleştirme yerine kardeşten bir eleman köke çıkarılır, kökten bir eleman aşağı indirilir: E yukarı çıkar, C aşağı iner. Sonuç: kök E L P T X; yapraklar A C, J K, N O, Q R S, U V, Y Z. Hiçbir düğüm $t - 1$'in altına düşmez.

Hoca D'nin silindiği durumu ağacın yüksekliğinin azaldığı tek durum olarak andı. Kuralların alt durumlarıyla birlikte 8 tane olduğunu, bunları sağlayan her tasarımın kabul edilebileceğini, ayrıntı isteyenlerin kitaba bakabileceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Şekildeki durum adları ders kitabındaki silme kurallarından gelir. Durum 1: anahtar yapraktadır, doğrudan silinir. Durum 2: anahtar iç düğümdedir; 2a'da önündeki çocukta en az t anahtar varsa öncül, 2b'de arkasındaki çocukta en az t anahtar varsa ardıl (successor) onun yerine geçer, 2c'de iki çocuk da $t - 1$ anahtarlıysa anahtar ve iki çocuk birleştirilir. Durum 3: anahtar bu düğümde değildir ve inilecek çocukta yalnız $t - 1$ anahtar vardır; 3a'da kardeşten ebeveyn üzerinden bir anahtar ödünç alınır, 3b'de çocuk bir kardeşle ve ebeveynden inen anahtarla birleştirilir.

11. Yığın (Heap)

Hoca sonraki derslerde Fibonacci yığınlarını (Fibonacci heap) anlatmak istediğini, ondan önce yığını hatırlatmak istediğini söyledi.

Yığın, neredeyse tam (nearly complete) bir ikili ağaçtır. Kesinlikle ikili arama ağacı değildir; "ikili" denmesinin sebebi her düğümün iki çocuğa ayrılmasıdır. Yığın özelliklerini taşıması gerekir. Öncelik kuyruklarının (priority queue) gerçeklenmesinde kullanılır: önceliği en yüksek ya da en düşük olan elemana en önce ulaşmak için. Bu yüzden max-heap ve min-heap özellikleri vardır ve yığın bunlardan birine uygun tasarlanır.

11.1 Dizide tutma

Fibonacci yığınları farklıdır, ama yığınları tutmak için daha çok diziler (array) kullanılır. Ağaç neredeyse tam olduğu için dizi, çocuklar ve ebeveynler arasında yeterince hızlı atlamayı sağlar; işaretçi aritmetiğinden ve onun getirdiği hesap yükünden kaçınılır.

Bir A dizisi bir yığını tutuyorsa (ekrandan, 01:44:20):

- Dizi $A[1 \ldots A.length]$ biçimindedir.

- $0 \leq A.heap_size \leq A.length$ olmak üzere A'nın `heap_size` alanı vardır.
- $A[1 \ldots A.heap_size]$ elemanları geçerli yığın elemanlarıdır. Kök her zaman $A[1]$'dir.

Hoca örnek bir ağaç çizip dizi görünümünü yanına yazdı. Ağaçta kök 16; ikinci seviyede 14 ve 10; üçüncü seviyede 8, 7, 9, 3; dördüncü seviyede 2, 4, 1 vardır. Düğümler soldan sağa 1'den 10'a numaralanır ve dizide ilgili indislere yerleşir:

İndis	1	2	3	4	5	6	7	8	9	10
$A[i]$	16	14	10	8	7	9	3	2	4	1

Bu bir max-heap'tir. Her alt ağaç kendi içinde düzenlidir, ama bütünde sıralı bir görüntü yoktur. Ağaç neredeyse dolu olduğu için dizide fazla boşluk olmaz; seyrek (sparse) değil, düzgün bir dizide tutulabilir.

11.2 Ebeveyn ve çocuklar

Hoca sınıfa i'inci elemanın ebeveynine ve çocuklarına nasıl ulaşılacağını sordu. Ebeveyne ulaşmak için indis ikiye bölünür. Sol çocuğa ulaşmak için ikiyle çarpılır, sağ çocuğa ulaşmak için ikiyle çarpılıp bir eklenir:

$$PARENT(i) = \lfloor i/2 \rfloor, \qquad LEFT(i) = 2i, \qquad RIGHT(i) = 2i + 1$$

Soru (hoca sordu): Bu işlemleri nasıl gerçeklersiniz? Fonksiyon mu yazarsınız, makro mu, doğrudan `2*i` mi? **Cevap:** Sürekli ikiye bölme ve ikiyle çarpma söz konusu olduğu için çarpma yerine bit işlemleri kullanılabilir: ebeveyni bulmak için bir kez sağa kaydırma (shift), sol çocuğu bulmak için bir kez sola kaydırma. Bunlar tek satırlık bit düzeyi (bitwise) işlemler ve makrolarla hızlıca yazılabilir.

11.3 Max-heap ve min-heap özelliği

Hoca özellikleri biçimsel olarak yazdı (ekrandan, 01:50:00):

- **Max-heap özelliği:** Kök dışındaki her i için $A[PARENT(i)] \geq A[i]$. Bütün ebeveynler kendi çocuklarından büyüktür. Max-heap'teki en büyük eleman köktür.
- **Min-heap özelliği:** Bunun tersidir: $A[PARENT(i)] \leq A[i]$. Min-heap'teki en küçük eleman köktür.

Silme ve ekleme işlemlerinde bu özelliklerin korunduğundan emin olmak gerekir. Bunun kontrolü için bir fonksiyon kullanılır.

12. Yığın Özelliğini Korumak: MAX-HEAPIFY

Tahtadaki başlık "Maintaining the heap property"dir (ekrandan, 01:52:08 ve 01:55:30):

```
MAX-HEAPIFY(A, i)
l = LEFT(i)
r = RIGHT(i)
if l <= A.heap_size and A[l] > A[i]
    largest = l
else largest = i
if r <= A.heap_size and A[r] > A[largest]
    largest = r
if largest != i
    exchange A[i] with A[largest]
    MAX-HEAPIFY(A, largest)
```

Fonksiyon A dizisinde i indisinden itibaren max-heap özelliğini korumak için çalışır. `LEFT(i)`, i'nin sol çocuğundan başlayan alt ağacı; `RIGHT(i)`, sağ çocuğundan başlayan alt ağacı verir. Fonksiyon bu iki alt ağacın kendi içlerinde max-heap olduğunu, köklerinin kendi alt ağaçlarının

en büyük elemanı olduğunu kabul eder; ama onların $A[i]$'ye göre durumunu bilmez ve bunu düzenler.

Yapılan basit bir karşılaştırmadır: üç eleman ($A[i]$, $A[l]$, $A[r]$) içinden en büyüğü bulunur. l yığın boyutunun içindeyse ve $A[l] > A[i]$ ise en büyük l'dir, değilse i'dir. Sonra bulunan en büyük, $A[r]$ ile karşılaştırılır; r daha büyükse `largest` r olur. En büyük bulunduktan sonra `largest`'ın i'den farklı olup olmadığına bakılır. Farklıysa $A[i]$ ile $A[largest]$ yer değiştirir ve fonksiyon özyinelemeli olarak `largest` için kendini çağırır.

12.1 Örnek: MAX-HEAPIFY(A, 2)

Hoca fonksiyonu tahtadaki ağaçta $i = 2$ için çalıştırdı (ekrandan, 01:55:30). Ağaç (anlatımdan ve son durumdan kurulmuştur; tahtada değerlerin üzerine yazıldığı için ilk hâl ayrıca okunamıyor):

İndis	1	2	3	4	5	6	7	8	9	10
$A[i]$	16	4	10	14	7	9	3	2	8	1

1. $i = 2$: 2'nin alt ağaçlarının (14 ve 7 kökleri) kurala uygun olduğu kabul edilir; ama 4'ün durumu bilinmez. 14, 4 ve 7 karşılaştırılır; en büyük 14'tür. 14 ile 4 yer değiştirir: 2. indiste 14, 4. indiste 4 olur.
2. $i = 4$: 4, 2 ve 8 karşılaştırılır; en büyük 8'dir. 8 ile 4 yer değiştirir: 4. indiste 8, 9. indiste 4 olur.
3. $i = 9$: Burası yapraktır; değiştirme yapılmaz.

Sonuç: 16, 14, 10, 8, 7, 9, 3, 2, 4, 1 (python ile doğrulandı). Bu, bir önceki bölümdeki örnek max-heap'tir.

13. Yığın Oluşturma: BUILD-MAX-HEAP

Bir A dizisi verilmiş, ama elemanların şu anki sıralanışı yığın özelliği taşımıyor olsun. Diziyi yığın özelliği taşır hâle getirmek için `MAX-HEAPIFY` kullanılır (ekrandan, 02:00:00):

```
BUILD-MAX-HEAP(A)
A.heap_size = A.length
for i = ⌊A.length/2⌋ downto 1
    MAX-HEAPIFY(A, i)
```

Soru (hoca sordu): $n = 10$ elemanlı bir yığında hangi elemanlar yapraktır, yani n'yi nasıl bir formülde kullanırsanız yaprakları gösterir? **Cevap:** 6, 7, 8, 9 ve 10. indisler yapraktır; özellikleri 1, 2, 3, 4, 5'ten sonra gelmeleridir. Yani $\lfloor n/2 \rfloor + 1, \lfloor n/2 \rfloor + 2, \ldots, n$ indisli elemanlar yapraktır; bunların dışında kalanlar iç düğümdür.

`BUILD-MAX-HEAP` yalnız iç düğümler üzerinde ilerler: en büyük indisli iç düğümden ($\lfloor n/2 \rfloor$) başlayıp 1'e kadar iner ve her seferinde `MAX-HEAPIFY`'ı çağırır.

> **Ek bilgi (derste anlatılmadı):** `MAX-HEAPIFY` en fazla ağacın yüksekliği kadar aşağı indiği için $O(\lg n)$ sürede çalışır. `BUILD-MAX-HEAP` bu fonksiyonu yaklaşık $n/2$ kez çağırsa da düğümlerin çoğu yaprağa yakın olduğundan toplam süresi $O(n \lg n)$ değil, daha sıkı bir analizle $O(n)$'dir.

13.1 Örnek

Hoca not alınmasını isteyerek şu diziyi verdi (ekrandan, 02:00:00):

İndis	1	2	3	4	5	6	7	8	9	10
$A[i]$	4	1	3	2	16	9	10	14	8	7

Dizinin ağaç görüntüsünde kök 4; ikinci seviyede 1 ve 3; üçüncü seviyede 2, 16, 9, 10; dördüncü seviyede 14, 8, 7 vardır. Yaprak olmayan düğümler, yarısından itibaren 5'ten 1'e (`downto`) işlenir: sırasıyla $i = 5, 4, 3, 2, 1$ için `MAX-HEAPIFY` çalışır.

1. $i = 5$: 16'nın kendi alt ağacında yalnız 7 vardır; 16 ve 7 karşılaştırılır, sorun yoktur.
2. $i = 4$: 2'nin çocukları 14 ve 8'dir; ikisi de 2'den büyüktür, büyük olanı 14'tür. 14 ile 2 yer değiştirir.
3. $i = 3$: 3'ün çocukları 9 ve 10'dur; en büyük 10'dur. 10 ile 3 yer değiştirir.
4. $i = 2$: Alt ağaçların kendi içlerinde kurala uyduğu, ama 14 ve 16'nın 1'den büyük olduğu görülür. 16 ile 1 yer değiştirir; 1, 5. indise iner. Orada 1'in çocuğu 7'dir; 7 ile 1 yer değiştirir. Bu adımdan sonra 2. indiste 16, 4. indiste 14, 5. indiste 7, 8. indiste 2, 9. indiste 8, 10. indiste 1, kökte 4 vardır.
5. $i = 1$: Özyinelemeli çağrılar sırasıyla şunu yapar: 16 ile 4 yer değiştirir (kök 16 olur); 4, 2. indise iner ve 14 ile yer değiştirir; 4, 4. indise iner ve 8 ile yer değiştirir. İşlem biter.

Sonuç dizi (ekrandan, 02:10:00):

İndis	1	2	3	4	5	6	7	8	9	10
$A[i]$	16	14	10	8	7	9	3	2	4	1

Her adımdaki yer değiştirmeler ve sonuç python ile doğrulandı.

Hoca yığınları böylece bitirdiğini, Fibonacci yığınlarından önce en küçük kapsayan ağaçtan (minimum spanning tree) da söz etmesi gerekeceğini söyledi.

Örnek Sorular

Soru 1 (ekrandan, 00:13:50): Kökünde M; iç düğümlerinde D H ve Q T X (tahtada Q T S yazılı); yapraklarında B C, F G, J K L, N P, R S, V W, Y Z bulunan ağaç, B-ağacı kurallarına göre t'nin hangi değerleri için geçerlidir?

> **Çözüm (derste verildi):** Kök dışındaki düğümlerde 2 ya da 3 anahtar vardır. $t = 2$ için 1-3, $t = 3$ için 2-5 anahtar gerekir; ikisi sağlanır. $t = 1$ olamaz ($t \geq 2$). $t = 4$ için en az 3 anahtar gerekir; 2 anahtarlı düğümler bunu bozar. Cevap: $t = 2$ ve $t = 3$.

Soru 2 (ekrandan, 00:20:00): $n \geq 1$ ise, yüksekliği h ve en küçük derecesi $t \geq 2$ olan n anahtarlı her B-ağacı için $h \leq \log_t \frac{n+1}{2}$ olduğunu gösteriniz.

> **Çözüm (derste verildi):** En az anahtarlı ağaç kurulur: kökte 1 anahtar ve 2 çocuk, diğer düğümlerde $t - 1$ anahtar ve t çocuk. Derinlik $i \geq 1$'de $2t^{i-1}$ düğüm vardır. $n \geq 1 + (t-1)\sum_{i=1}^{h} 2t^{i-1} = 2t^h - 1$; buradan $n + 1 \geq 2t^h$ ve $h \leq \log_t \frac{n+1}{2}$.

Soru 3 (derste sözlü, 00:31:19): `B-TREE-SEARCH` yordamının karmaşıklığı nedir?

> **Çözüm (derste verildi):** Düğüm içi `while` taraması en fazla $2t - 1$ anahtar, yani $O(t)$; özyinelemeli iniş ağacın derinliği kadar, yani $O(\log_t n)$. Toplam $O(t \log_t n)$.

Soru 4 (ekrandan, 01:24:20): $t = 3$ olan, kökünde G M P X ve yapraklarında A C D E, J K, N O, R S T U V, Y Z bulunan B-ağacına sırasıyla B, Q, L ve F ekleyiniz.

> **Çözüm (derste verildi):** B: A B C D E. Q: R S T U V bölünür, T köke çıkar (G M P T X); Q R S. L: kök dolu, bölünür; P yeni kök, çocukları G M ve T X; J K L. F: A B C D E bölünür, C çıkar (C G M); A B ve D E F. Son ağaç: kök P; C G M, T X; A B, D E F, J K L, N O, Q R S, U V, Y Z (python ile doğrulandı).

Soru 5 (ekrandan, 01:35:05): $t = 3$ olan, kökünde P; iç düğümlerinde C G M ve T X; yapraklarında A B, D E F, J K L, N O, Q R S, U V, Y Z bulunan B-ağacından sırasıyla F, M, G, D ve B silininiz.

> **Çözüm (derste verildi):** F: yapraktan silinir (D E). M: öncül L yerine geçer (C G L; J K). G: D E ve J K birleşir (C L; D E J K). D: C L, P ve T X birleşir, kök boşalır ve yükseklik azalır (C L P T X; E J K). B: E köke çıkar, C aşağı iner (E L P T X; A C, J K).

Soru 6 (derste sözlü, 01:45:31): Dizide tutulan bir yığında ebeveyn, sol çocuk ve sağ çocuk indislerini hesaplayan işlemleri nasıl gerçeklersiniz?

> **Çözüm (derste verildi):** $PARENT(i) = \lfloor i/2 \rfloor$, $LEFT(i) = 2i$, $RIGHT(i) = 2i + 1$. İkiye bölme ve ikiyle çarpma, sağa ve sola bir bit kaydırmayla yapılabilir; tek satırlık makrolarla hızlıca yazılır.

Soru 7 (derste sözlü, 01:56:50): $n = 10$ elemanlı bir yığında hangi elemanlar yapraktır?

> **Çözüm (derste verildi):** 6, 7, 8, 9, 10. indisler; genel olarak $\lfloor n/2 \rfloor + 1, \ldots, n$.

Soru 8 (ekrandan, 01:55:30): $A = \langle 16, 4, 10, 14, 7, 9, 3, 2, 8, 1 \rangle$ için `MAX-HEAPIFY(A, 2)` çalıştırınız.

> **Çözüm (derste verildi):** 4 ile 14 yer değiştirir, sonra 4 ile 8 yer değiştirir; 9. indis yapraktır. Sonuç: 16, 14, 10, 8, 7, 9, 3, 2, 4, 1 (python ile doğrulandı). Başlangıç dizisi anlatımdan kurulmuştur.

Soru 9 (ekrandan, 02:00:00): $A = \langle 4, 1, 3, 2, 16, 9, 10, 14, 8, 7 \rangle$ dizisine `BUILD-MAX-HEAP` uygulayınız.

> **Çözüm (derste verildi):** $i = 5$: değişiklik yok. $i = 4$: 2 ile 14. $i = 3$: 3 ile 10. $i = 2$: 1 ile 16, sonra 1 ile 7. $i = 1$: 4 ile 16, 4 ile 14, 4 ile 8. Sonuç: 16, 14, 10, 8, 7, 9, 3, 2, 4, 1 (python ile doğrulandı).

Dersin Toparlaması

Dersin büyük kısmı B-ağaçlarına ayrıldı. Önce bir B-ağacının bütün işlemlerde koruması gereken özellikleri yazıldı: `x.n` anahtar ve `x.n + 1` çocuk, düğüm içinde sıralı anahtarlar, anahtarların alt ağaç aralıklarını ayırması, bütün yaprakların aynı derinlikte olması ve en küçük derece t'ye bağlı en az $t - 1$, en fazla $2t - 1$ anahtar sınırı. Bu sınırların yüksekliği $h \leq \log_t \frac{n+1}{2}$ ile kısıtladığı, en az anahtarlı ağaç üzerinden kanıtlandı. Ardından arama ($O(t \log_t n)$), boş ağaç oluşturma, dolu bir çocuğu ortancasını ebeveyne çıkararak bölen `B-TREE-SPLIT-CHILD` ve inerken dolu düğümleri bölerek anahtarı yaprağa ekleyen `B-TREE-INSERT` / `B-TREE-INSERT-NONFULL` sözde kodlarıyla anlatıldı ve $t = 3$ için bir ekleme örneğiyle izlendi. Silme, düğümlerin $t-1$'in altına düşmemesi ilkesiyle, kitap şekli üzerinden öncülle değiştirme, birleştirme ve kardeşten ödünç alma durumlarıyla gösterildi.

Son kısımda Fibonacci yığınlarına hazırlık olarak ikili yığın hatırlatıldı: dizide tutulması, ebeveyn ve çocuk indisleri, max-heap ve min-heap özellikleri, özelliği koruyan `MAX-HEAPIFY` ve bir diziyi yığına çeviren `BUILD-MAX-HEAP`. Hoca Fibonacci yığınlarından önce en küçük kapsayan ağaçlardan da söz edeceğini belirtti.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.